

p3.py

→ find performance

import lib

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
```

load the data

```
data = pd.read_csv("car_condition.csv")
data.columns = ["buying", "maintenance", "doors", "persons", "lug_boot", "safety", "class"]
print(data)
```

check and handle null data

```
print(data.isnull().sum())
```

features and target

```
features = data.drop(["class"], axis="columns")
target = data["class"]
```

handle cat data

```
nfeatures = pd.get_dummies(features)
print(nfeatures)
```

train and test

```
x_train, x_test, y_train, y_test = train_test_split(nfeatures, target)
```

model

```
model = DecisionTreeClassifier()
model.fit(x_train, y_train)
```

performance

```
print(data["class"].value_counts())
y_pred = model.predict(x_test)
cr = classification_report(y_test, y_pred)
print(cr)
```


P4.py **-> find important features**

import lib

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
```

load the data

```
data = pd.read_csv("car_condition.csv")
data.columns = ["buying", "maintenance", "doors", "persons", "lug_boot", "safety", "class"]
print(data)
```

check and handle null data

```
print(data.isnull().sum())
```

features and target

```
features = data.drop(["class"], axis="columns")
target = data["class"]
```

handle cat data

```
nfeatures = pd.get_dummies(features)
print(nfeatures)
```

train and test

```
x_train, x_test, y_train, y_test = train_test_split(nfeatures, target)
```

model

```
model = DecisionTreeClassifier()
model.fit(x_train, y_train)
```

feature importance

```
fi = model.feature_importances_
print(fi)
co = nfeatures.columns
print(co)
```

```
plt.figure(figsize=(12, 5))
plt.barh(co, fi)
plt.show()
```


P5.py → reduce the dimension

import lib

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
```

load the data

```
data = pd.read_csv("car_condition.csv")
data.columns = ["buying", "maintenance", "doors", "persons", "lug_boot", "safety", "class"]
print(data)
data.drop(["buying", "maintenance", "doors", "lug_boot"], axis="columns", inplace=True)
```

check and handle null data

```
print(data.isnull().sum())
```

features and target

```
features = data.drop(["class"], axis="columns")
target = data["class"]
```

handle cat data

```
nfeatures = pd.get_dummies(features)
print(nfeatures)
```

train and test

```
x_train, x_test, y_train, y_test = train_test_split(nfeatures, target)
```

model

```
model = DecisionTreeClassifier()
model.fit(x_train, y_train)
```

performance

```
print(data["class"].value_counts())
y_pred = model.predict(x_test)
cr = classification_report(y_test, y_pred)
print(cr)
```


